ФСП Карьера

Сопроводительная документация решения по специальному треку ФСП. Архитектура, механики, процедура валидации, интеграции и инструкция запуска.

Дата документа: 11 октября 2026 года.

ФСП Карьера - веб-платформа обратного найма ИТ-специалистов с подтверждением квалификации. Документ описывает пользовательские сценарии, компоненты, алгоритмы оценки и подбора, результаты проверок, интеграционные контракты и воспроизводимый запуск.

Исходный код: https://github.com/m1kkon/fsp-career. Прототип: https://fspcareer.space. Версия опубликованного приложения доступна в поле revision ответа /api/health.

1. Назначение и функциональная архитектура

Платформа реализует обратный найм ИТ-специалистов. Кандидат подтверждает квалификацию тестированием, а работодатель описывает потребность, получает объяснимую подборку и первым направляет предложение с зарплатой. Классические вакансии и самостоятельные отклики существуют как дополнительный канал, не заменяя обратный найм.

1.1. Сценарий кандидата

Кандидат регистрируется с подтверждением email, заполняет резюме и согласия, проходит анкету по отрасли, специализации и формату работы, выбирает предполагаемый грейд и проходит опубликованные модули роли. Результат теста определяет подтверждённую категорию для каталога работодателя. Предполагаемый, подтверждённый и потенциальный грейд отображаются как разные сущности: сильный результат на Middle не делает профиль автоматически Senior.

Резюме редактируется вручную: ФИО, контакты, стек, опыт, роли, навыки, языки, проекты и ссылки. Сервер формирует стандартизированный PDF. Частичное сохранение меняет только переданные поля; отсутствие поля не является командой удалить раздел. Для ввода резюме используется структурированная ручная форма; PDF является форматом выгрузки профиля.

Кандидат видит только свои приглашения, отклики, ответы на задания и тестовые результаты. Он принимает или отклоняет предложение, управляет публикацией профиля и историей ФСП. Сервис работает и без ФСП ID, и с пустой историей событий.

1.2. Сценарий работодателя

Работодатель заполняет профиль компании, описывает команду и потребность обычным текстом, затем подтверждает извлечённые ограничения: специализацию, минимальный грейд, обязательный и желательный стек, опыт, формат, город, зарплатный бюджет. Требование не считается обязательным только потому, что парсер распознал слово: пользователь проверяет форму.

Каталог показывает категории и кандидатов с краткими причинами соответствия. Работодатель уточняет запрос и сохраняет подборки. Приглашение можно направить без публикации вакансии. Компания, текст предложения, контакт компании и положительный диапазон зарплаты от-до в рублях доступны кандидату до согласия на контакт.

Работодатель публикует и редактирует вакансии, получает самостоятельные отклики, создаёт короткие практические задания, проверяет ответы и оставляет обратную связь. Первоначальное условие сохраняется вместе с ответом: последующее редактирование задания не переписывает историю решения.

1.3. Согласованный контакт и видимость

До принятия приглашения или самостоятельного отклика контакт кандидата скрыт от конкретного работодателя. Раскрытие не является глобальной публикацией контактов. На сервере проверяются роль, владелец и участники диалога; интерфейсные ограничения не являются единственным средством защиты. Статусы отправки, просмотра и решения доступны обеим сторонам по своим записям.

2. Компоненты и хранение

Компонент	Реализация и назначение
Frontend	Next.js и React; страницы app/, кабинет public/mvp.js, тестовый интерфейс public/assessments.js, адаптивный CSS.
HTTP/API	Обработчики app/api/, внутренняя API кабинета и публичная API v1 с ключами. Валидация запросов, аутентификация и серверные проекции данных.
Бизнес-логика	lib/service.mjs: роли, профиль, анкета, потребности, предложения, вакансии, задания, согласия и действия пользователя.
Тестирование	lib/assessments.mjs, адаптивные спецификации, оценивание, банк вопросов и версионные снимки попыток.
Подбор	lib/matching.mjs, lib/ats.mjs, lib/resume-fit.mjs: ограничения, NLP-извлечение, объяснения и рейтинг.
База	PostgreSQL 17; аккаунты, сессии, API-ключи, бизнес-записи JSONB, спецификации и попытки. Эксплуатационные миграции в db/migrations/.
Кодинг	Изолированный Docker runner либо приватный HTTP worker; сетевые и ресурсные ограничения, таймауты.
Почта	Resend и зашифрованная очередь; подтверждение email и восстановление доступа.
Эксплуатация	Docker Compose, Caddy, GitHub Actions, GCP VM; резервное копирование и инструкции восстановления.

Приложение построено как модульный монолит с явным разделением интерфейса, API, бизнес-логики и хранения. API, правила подбора и исполнительный сервис можно масштабировать отдельно. Транзакции защищают изменения бизнес-записей; исполнение пользовательского кода происходит вне общей блокировки базы, с проверкой актуальности попытки перед сохранением результата. Повторная доставка того же ID и исходного кода идемпотентна; изменение исходника или закрытие попытки не позволяет применить устаревшее решение. Параллельное выполнение на worker ограничено четырьмя запросами, ответы worker валидируются сервером.

Compose ограничивает память основных сервисов: приложение 1536 MiB, PostgreSQL 768 MiB, Caddy 256 MiB, одноразовая миграция 256 MiB. Это лимиты конфигурации, не измеренное потребление под нагрузкой. Отдельному sandbox нужны собственные ресурсы; ёмкость сервера определяется с учётом числа одновременных исполнений и результатов нагрузочных измерений.

3. Механика тестирования и категоризации

3.1. Разные формы при сопоставимых правилах

Для основных ИТ-направлений задан справочник ролей и обязательных модулей. В банке 2359 записей, из них 559 задач с кодом, включая 10 выведенных из использования. Число записей не равно числу одновременно активных заданий. Структурный аудит проверяет ID, варианты, ключи, ограничения и заданные проверки, а не исполняет эталонные решения.

Модуль состоит из двух частей; используются выборки из семейств задач и адаптивные ветки. Шаблон, версия правил, состав предъявленной формы и порядок фиксируются сервером для начатой попытки. Ключи ответов не выдаются чужим пользователям. Ежедневные пулы, версия банка, разнообразие форм и кодинг затрудняют распространение единого списка ответов. Общие модули позволяют сопоставлять одинаковые компетенции между ролями.

Разнообразие форм и скрытые проверки уменьшают пользу заранее распространённых ответов. Внешняя помощь и утечка банка остаются ограничениями подхода; биометрический прокторинг не используется. Статистический анализ аномалий предусмотрен для дальнейшего развития системы контроля.

3.2. Оценка, подтверждение и повторное прохождение

Политика rasch-theory-practice-v3 объединяет 60% теории и 40% практики с поправкой на сложность предъявленных заданий. Кроме общего порога применяются минимумы обеих частей и требование полностью решённой задачи нужной сложности по обязательным модулям. Сырой процент решения кодинга показывается отдельно от нормализованного результата.

Грейд не понижается принудительно. Не подтвердив выбранный уровень, кандидат получает объяснение и возможность выбрать следующий тест на подходящий уровень; уверенное прохождение даёт рекомендацию попробовать выше. Потенциальный более высокий уровень не заменяет подтверждённый. Смена подтверждённого грейда ограничена интервалом 90 дней. Старые результаты и начатые попытки сохраняют исходные спецификации, не пересчитываются новым алгоритмом задним числом.

Категория работодателя строится по специализации и подтверждённому тестом грейду. Стек и стаж резюме остаются заявленными данными и не превращаются в подтверждённые компетенции. Политика, время, баллы, рекомендации и история доступны владельцу профиля.

Теория оценивается через модель с логистической вероятностью решения: logistic(x) = 1 / (1 + exp(-x)). Для предъявленных заданий с якорями сложности b численно определяется параметр θ из уравнения sum logistic(θ - b) = число правильных ответов; итог теории равен 100 x logistic(θ). Практика использует аналогичное уравнение с частичным зачётом скрытых проверок и собственными якорями сложности. Итог объединяет 60% теории и 40% практики, после чего применяются минимумы обеих частей и ограничения задач соответствующего уровня. Якоря сложности являются предварительной авторской калибровкой, а не параметрами, оценёнными на независимой популяции людей.

3.3. Вакансии и регулярная практика

Связь роли, компетенций, модулей и банка позволяет показать выбор заданий на нескольких описаниях вакансий. Примеры из текущего справочника ролей lib/question-bank.mjs приведены ниже; каждый набор включает интеграционное задание, связывающее навыки роли. Для каждой опубликованной роли фактический состав определяется спецификацией, а не произвольным генерированием вопросов нейросетью.

Пример потребности	Модули роли и интеграция
Middle backend: Python, PostgreSQL, сервисы и API	backend-developer: python, sql, backend; integration-backend.
Frontend: JavaScript, React, браузерный интерфейс	frontend-developer: javascript, browser, react; integration-frontend.
DevOps: инфраструктура, контейнеры, Python-автоматизация	devops-engineer: infrastructure, python; integration-devops.

«Задание недели» вычисляется сервером с интервалом 7 дней; предложение стабильно в рамках периода и не включает уже отвеченные задания. Периодические короткие задания компаний являются отдельной практикой: предложение учитывает специализацию и уже отвеченные задачи, кандидат видит историю, компания даёт оценку и обратную связь. Проверенная практика обновляет сигнал актуальности профиля, но не присваивает

подтверждённый грейд и не подменяет основной тест. Изменённое/снятое задание не уничтожает исходное условие ранее отправленного ответа.

4. Механика подбора и NLP

Основной порядок опирается на подтверждённую категорию, а не на самостоятельно записанный Senior в резюме. Требования работодателя проверяются с исходами pass, unknown, fail; неизвестная готовность к переезду не считается доказанным отказом. Минимальный грейд Middle допускает подтверждённого Senior. Зарплатные диапазоны сопоставляются по пересечению, включая совпадающую границу.

Внутри соответствующих категорий рейтинг отражает потребность: специализация до 25, минимальный грейд 15, стек 12, опыт и проекты 20, тест 12, ФСП 6, формат 4, зарплата 4, предметная область 2; максимум 100. Это объясняемые веса MVP, не вероятность успешного найма. Рядом с баллом приводятся причины и пропуски данных. Без заданной потребности показывается отдельная сила подтверждённого профиля: тесты и ФСП, до 18.

Словарная NLP-нормализация технологий и правиловой анализ текста извлекают стек, типы задач, опыт, отрицания, пожелания и планы. «Не работал с X» и «планирую изучить X» не подтверждают практический опыт. Цитаты служат основанием объяснения. Уточнение формы явно заменяет извлечённые поля, в том числе очищает их. TF-IDF используется в benchmark как отдельный baseline, а не скрытое слагаемое выдачи. Обученная нейросетевая модель и embeddings в рабочем ранжировании не используются.

5. Процедура валидации и результаты

Процедура объединяет программные и интеграционные проверки, контролируемый симулятор с авторской эталонной разметкой и отдельный benchmark реальных текстов. Для каждого измерения различаются проверяемая реализация, источник данных, объём выборки и область применимости результата.

5.1. Программные и интеграционные проверки

В GitHub Actions на ревизии 72ffb4c выполнены **427 тестов: 427 прошли, 0 ошибок, 0 пропусков**. Журнал проверки. Проверки используют PostgreSQL 17 и реальный Docker sandbox. Production-сборка Next.js, TypeScript и ESLint завершились успешно; ESLint содержит 23 существующих предупреждения, ошибок нет.

Набор покрывает роли и владельцев записей, подтверждение email, приватность контактов, приглашения и отклики, сохранение профиля, диапазоны зарплат, категорию, историю тестов, конкурентные операции, API, таймауты и исполнение кода в изоляции. Структурный аудит банка проверяет 2359 записей, 559 кодовых задач, 10 выведенных из использования и 2881 заданную проверку результата. Структурный аудит сам по себе не является исполнением всех эталонных решений банка.

В локальном браузерном стенде пройден сценарий: публикация задания работодателем → предложение по специализации кандидата → ответ → редактирование условия → проверка первоначального снимка → обратная связь → сохранённое условие и feedback у кандидата. При размере 390 x 844 горизонтальное переполнение отсутствует. Этот результат относится к указанному локальному сценарию, а не к доставке production-почты или внешнему реестру ФСП.

В том же CI проверено конкурентное сохранение профилей во время ожидания judge: 50 аккаунтов и 100 операций сохранения. Сохранения завершились за 206 мс, пока judge оставался в состоянии ожидания. Этот сценарий проверяет сервисные транзакции PostgreSQL, а не HTTP-пропускную способность. Production p95, максимальная пропускная способность и SLA этим тестом не установлены.

Языковые и matching-регрессии: 69 тестов прошли, 0 ошибок, 0 пропусков; в том числе 25 новых языковых случаев и 16 ранее существовавших регрессий. Машинные результаты и параметры экспериментов хранятся в docs/current-product-validation.json и docs/matching-current-validation.json.

5.2. Синтетическая разметка и стабильность грейда

Для виртуальных профилей разных уровней задана авторская модель вероятности решения задач. Проверяются совпадение ожидаемого и полученного грейда, повторяемость форм, завышение и распознавание Middle; метрики разбиты по уровню. Параметры, формы, агрегаты и хеши воспроизводимого прогона сохранены в docs/current-product-validation.json.

Показатель текущего синтетического holdout	Наблюдённый результат
Размер	1200 виртуальных профилей; два прохождения, 2400 результатов
Совпадение с авторским эталоном	93,17%
Совпадение грейда при повторном прохождении	87,25%
Распознавание Middle	86,50%
Завышение относительно эталона	0,67%
Проверки устойчивости модели	Пороговые проверки development/holdout; отдельные случаи только теории, только кодинга и случайного выбора выполнены

На 24 синтетических потребностях и 480 парах P@5 подбора 97,5%; проверка допустимости и суммы объяснений 100%, недопустимых профилей в первой пятёрке 0. На этой авторской разметке задана ожидаемая релевантность; это не точность на реальном рынке. Синтетический holdout отделяет роли/семейства сценариев и формы, но использует ту же авторскую модель ответов. Кодинг моделируется доверенными снимками судьи, не исполняется в этом эксперименте.

Числа относятся к наблюдённому локальному симулятору, не к людям и не к выпущенному production-релизу. Монотонные вероятности виртуальных уровней помогают выявлять ошибки реализации, но не доказывают содержательную сложность банка или реальную психометрическую дискриминативность. Происхождение разметки раскрывается в отчётах. Предыдущие измерения сохранены в docs/grade-hardening.md и docs/synthetic-validation.md.

5.3. Реальные тексты и сравнение baseline

Talent-Match: 29 вакансий, 656 пар; поддерживаемые направления - 13 вакансий. Development: 9 вакансий и 246 пар; ранее известный holdout: 4 вакансии и 102 пары. Исходный commit и SHA-256, seed разбиения и хеши реализации сохранены в docs/matching-current-validation.json. Разбиение не пересекается по вакансиям и резюме.

Метрика P@5	Development	Ранее известный holdout
Текущий правиловой подбор	40,0%	35,0%
TF-IDF baseline	33,3%	25,0%
Stack baseline	31,1%	30,0%
Случайный порядок, ожидаемое значение	31,6%	23,4%

На holdout nDCG@5 продукта 0,432, MRR 0,583. Это воспроизведение на уже изучавшейся выборке, не новый независимый финальный тест. Языковые исправления не дали роста агрегатного P@5; повышение качества подбора на этих данных не заявляется. Метки confirmed/failed являются сигналом скрининга, не фактом найма. Источник не содержит подтверждённых грейдов, тестов и ФСП, поэтому такие баллы при оценке этих данных не начисляются. Этот benchmark проверяет ранжирование доступных текстов, а не полный каталог подтверждённых категорий.

5.4. Воспроизведение

```
npm test
npm run bank:audit
node scripts/benchmarks/synthetic-validation.mjs --label current --split all --output output/validation/synthetic-current
node scripts/benchmarks/harden-grades.mjs --assert --output output/validation/grade-current
node scripts/benchmarks/talentmatch.mjs --input /absolute/path/source.json --output /local/results
```

Параметры симулятора описаны в docs/validation.md для конкретной версии: отчёты сопровождаются хешами исходников. Для Talent-Match нужен исходный файл указанной ревизии; он не заменяется свежим случайным источником. Публикуемый результат содержит только агрегаты и хеши, без исходных резюме и контактов.

6. Предполагаемая интеграция ФСП

Текущий lib/fsp.mjs - демонстрационный серверный адаптер на собственных фикстурах. Источник помечен demo в интерфейсе и PDF. Есть 44 тестовых ID; несколько возвращают пустую историю. Наличие ID не является подтверждением принадлежности реальному участнику. Отсутствие истории не блокирует регистрацию, тест, поиск и контакт; бонус ФСП отсутствует, остальные факторы сохраняются.

Предлагаемая запись достижения: ID события, дисциплина, год, результат, уровень, командный/индивидуальный формат, источник и время проверки. Из публичной проекции исключается неподтверждённая ручная подмена достижений. История усиливает рейтинг до 6 баллов, не назначает грейд и не отменяет согласия владельца.

Будущий адаптер получает контракт и основание доступа от ФСП. После OIDC/Keycloak-проверки владения идентификатором сервер нормализует достижения и хранит происхождение, дату синхронизации и состояние актуальности. При временном сбое последняя подтверждённая история помечается устаревшей: ошибка источника не превращается в «достижений нет». Нужны отзыв согласия, журнал изменений, повторы синхронизации и тесты пустого/недоступного реестра. Реальные endpoint, scopes и форматы ответов уточняются после предоставления контракта реестра ФСП.

7. Концепции ATS и защиты от фиктивных предложений

7.1. ATS

Граница интеграции - корпоративный API-ключ с минимальными правами и конкретной компанией. До согласия кандидата ATS получает только разрешённую обезличенную проекцию. После принятия предложения или самостоятельного отклика экспортируются согласованные контакты, предложение и статус; сопоставление использует внешние ID, а не email как глобальный ключ.

Промышленный коннектор предусматривает очередь исходящих событий, подпись HMAC, eventId и ключ идемпотентности, экспоненциальные повторы, dead-letter очередь, журнал доставки и endpoint повторной синхронизации. Входящие изменения проверяют компанию, допустимый переход статуса и версию записи; устаревшее событие не откатывает новый статус. Отзыв ключа/согласия запрещает следующие выгрузки. Минимальный контракт событий: invitation.sent/viewed/accepted/declined, application.created/statusChanged и consent.revoked. Публичный API v1 реализован. Внешний ATS-коннектор и очередь webhook описаны как архитектура следующего этапа; промышленная интеграция с конкретной ATS не подключена.

7.2. Антифрод работодателей

Предлагается поэтапная схема: подтверждённый email, доказательство владения корпоративным доменом, проверка реквизитов при повышении лимитов, прозрачная метка уровня проверки. Риск оценивается по массовым однотипным предложениям, аномальным условиям, повторным жалобам и изменениям домена/контакта. Сигнал не означает автоматическое признание мошенничества: спорные случаи проходят ручную модерацию с возможностью обжалования.

Ограничения: лимиты приглашений и публичных публикаций, запрет приглашений от заблокированной компании, кнопка жалобы с сохранением доказательств, журнал решений и срок пересмотра. Сбор минимальных данных и контролируемый доступ к жалобам предотвращают превращение антифрода в публичный список обвинений. Верификация юридических лиц и очередь модерации описаны как следующий этап, не как уже подключённый сервис.

8. API и безопасность

Публичная API: https://fspcareer.space/api/v1. Руководство: /api-docs; машинный контракт OpenAPI 3.1: /api-v1.openapi.json. Внутренний контракт кабинета: /openapi.json. Все актуальные схемы и коды ошибок публикуются в репозитории; генератор scripts/generate-public-api.mjs поддерживает согласованность руководства и схем.

API-ключи создаются после входа с подтверждённым email, имеют права чтения/записи и срок 30/90 дней; полный ключ показывается один раз. Передавайте ключ в Authorization: Bearer KEY, храните его на сервере интеграции. Роль, владелец и доступ к контакту проверяются при каждом запросе.

Группа	Назначение и контракт
/api/session	Регистрация, вход, подтверждение, выход и восстановление; cookie-сессия.
/api/state, /api/action	Кабинет: собственное состояние и разрешённые бизнес-действия, включая тестирование и задачи.
API v1: /state, /me, /candidates	Состояние, профиль владельца, разрешённый каталог и объяснимые фильтры.
API v1: /jobs, /needs, /invites, /applications	Вакансии, подборки, предложения и статусы; ограничения доступа сохраняются.
API v1: /tasks, /submissions, /messages	Задания, ответы и переписка только доступных участнику записей.
/api/health	Статус и ревизия релиза; не публикует секреты и персональные данные.

Успешные ответы - JSON UTF-8, ошибки различают некорректный ввод, отсутствие входа, запрет, отсутствующий ресурс и конфликт состояния. Точные тела, обязательные поля и числовые HTTP-коды смотрите в OpenAPI и docs/api.md; документация не заменяет runtime-проверку. Приглашения и вакансии требуют целочисленный положительный диапазон зарплат; неоднозначные десятичные суммы отклоняются.

Пароли: scrypt и индивидуальная соль; сессии и одноразовые токены хешируются; cookie HttpOnly/Secure в production; отправляемые почтовые данные шифруются. Сервер проверяет Origin, доступ к владельцу и согласия. Пользователь может ограничить публикацию и убрать связь ФСП. Указанные механизмы являются техническими мерами защиты. Юридическое соответствие 152-ФЗ, сведения об операторе, размещение и сроки хранения зависят от фактической модели эксплуатации; результаты автоматических проверок не являются юридическим заключением.

9. Версии библиотек и компонентов

Версии ниже берутся из package-lock.json; диапазоны package.json не подменяют точную разрешённую версию. Полный lockfile предоставляется с исходниками. Сторонние компоненты сохраняют собственные лицензии. Включённые шрифты Liberation Serif распространяются по SIL Open Font License 1.1; лицензия находится в vendor/resume-fonts.LICENSE.txt.

Библиотека / компонент	Точная версия
@base-ui/react	1.7.0
@hookform/resolvers	5.7.1
@pdf-lib/fontkit	1.1.1
@shadcn/react	0.3.0
class-variance-authority	0.7.1
clsx	2.1.1
cmdk	1.1.1
date-fns	4.4.0
drizzle-orm	0.45.2
embla-carousel-react	8.6.0
input-otp	1.4.2
lucide-react	1.31.0
next	16.3.8
next-themes	0.4.6
parse5	8.0.0
pdf-lib	1.17.1
pg	8.23.1
postcss	8.5.23
radix-ui	1.6.7
react	19.2.6
react-day-picker	10.0.1
react-dom	19.2.6
react-hook-form	7.85.0
react-resizable-panels	4.12.2
recharts	3.8.0
sonner	2.0.8

Библиотека / компонент	Точная версия
tailwind-merge	3.6.0
vaul	1.1.2
zod	3.25.76
@tailwindcss/postcss	4.2.1
@types/node	22.19.19
@types/pg	8.23.1
@types/react	19.2.14
@types/react-dom	19.2.3
drizzle-kit	0.31.10
eslint	9.39.4
eslint-config-next	16.3.8
json-rpc-2.0	1.7.1
raw-body	3.0.2
tailwindcss	4.2.1
tw-animate-css	1.4.0
typescript	5.9.3
Node.js	24 в CI и Docker; локальная среда валидации 24.19.0; минимальная поддерживаемая версия 22.13.0
PostgreSQL	17-alpine, pinned digest в compose.yaml
Caddy	2-alpine, pinned digest в compose.yaml
Python для документации	3.10+
reportlab	4.4.9 в среде сборки; install команда допускает совместимую 4.x

10. Сборка, локальный запуск и развёртывание

10.1. Локальная машина

Требуются Node.js >=22.13.0 (CI использует 24), npm, PostgreSQL 17; для выполнения кода Docker или приватный worker. Для PDF-документации дополнительно Python 3.10+ и reportlab, для визуальной проверки Poppler.

```
git clone https://github.com/m1kkon/fsp-career.git
cd fsp-career
npm ci --include=dev --include=optional
```

Создайте отдельную БД и игнорируемый .env.local; реальные секреты не коммитятся. MAIL_TOKEN_KEY - 64 hex-символа; почтовый отправитель должен быть подтверждён в Resend.

```
DATABASE_URL=postgres://USER:PASSWORD@127.0.0.1:5432/fsp_local
```

```
APP_URL=http://localhost:3000
RESEND_API_KEY=YOUR_SENDING_KEY
EMAIL_FROM=FSP Career <no-reply@YOUR_VERIFIED_DOMAIN>
MAIL_TOKEN_KEY=YOUR_64_HEX_CHARACTERS

node --env-file=.env.local scripts/migrate.mjs
npm run dev
node --env-file=.env.local scripts/send-mail.mjs
```

Откройте http://localhost:3000. Последняя команда делает один проход почтовой очереди; для непрерывной доставки нужен планировщик. Не подставляйте production-базу в тесты. Подтверждение email выполняется по одноразовой ссылке через настроенную почтовую службу.

10.2. Проверки и production build

```
npm run build
npx tsc --noEmit
npm run lint
npm test
npm run bank:audit
```

TypeScript запускается после сборки клиентского манифеста. PostgreSQL-тесты требуют TEST_DATABASE_URL отдельной базы с окончанием _test; реальные sandbox-тесты требуют Docker и CODE_SANDBOX_TESTS=1. Конкретные команды образа и worker приведены в docs/coding-and-role-grades.md. Без этих компонентов соответствующие проверки помечаются как пропущенные.

10.3. Docker и сервер

Создайте защищённый .env на сервере: SITE_DOMAIN, POSTGRES_PASSWORD, Resend, EMAIL_FROM, MAIL_TOKEN_KEY; при отдельном worker также CODE_SANDBOX_URL и CODE_SANDBOX_TOKEN. Полный список и порядок GCP/IAP описаны в docs/deployment.md.

```
docker compose build
docker compose up -d
docker compose ps
curl --fail https://YOUR_DOMAIN/api/health
```

Compose сначала ждёт PostgreSQL, выполняет миграции, затем запускает приложение и Caddy. Ответ /api/health идентифицирует установленную ревизию. Эксплуатация предусматривает резервное копирование PostgreSQL, проверку восстановления на отдельной базе, аутентификацию приватного worker и мониторинг доставки почты. Выполнение миграций и обновление приложения входят в отдельную процедуру выпуска.